Module 2: Designing OpenStack Cloud Architectural Consideration – OpenStack - The new data center paradigm – OpenStack logical architecture - Nova - Compute Service.

Designing OpenStack Cloud Architecture

OpenStack – an open source software project

OpenStack is an open source software project which has a huge number of contributors from a wide range of organizations. OpenStack was originally created by NASA and Rackspace. Rackspace is still a significant contributor to OpenStack, but these days contributions to the project come from a wide array of companies, including the traditional open source contributors (Red Hat, IBM, and HP) as well as companies which are dedicated entirely to OpenStack (Mirantis, and CloudBase). Contributions come in the form of drivers for particular pieces of infrastructure (that is, Cinder block storage drivers or Neutron SDN drivers), bug fixes, or new features in the core projects.

OpenStack is governed by a foundation. Membership in the foundation is free and open to anyone who wishes to join. There are currently thousands of members in the foundation. Leadership on technical issues is provided by a thirteen-member technical committee, which is generally elected by the individual members. Strategic and financial issues are decided by a board of directors, which includes members appointed by corporate sponsors and elected by the individual members.

For more information on joining or contributing to the OpenStack Foundation, refer to http://www.openstack.org/foundation.

OpenStack is written in the Python programming language and is usually deployed on

the Linux operating system. The source code is readily available on the Internet and commits are welcome from the community at large. Before code is committed to the project, it has to pass through a series of gates, which include unit testing and code review.

For more information on committing code to OpenStack, refer to https://wiki.openstack.org/wiki/How_To_Contribute.

OpenStack – a private cloud platform

OpenStack provides the software modules necessary to build an automated private cloud platform. While OpenStack has traditionally been focused on providing Infrastructure as a Service capabilities in the style of Amazon Web Services, new projects have been introduced lately, which begin to provide capabilities which might be associated more with Platform as a Service. This book will focus on implementing the core set of OpenStack components described as follows.

The most important aspect of OpenStack pertaining to its usage as a private cloud platform is the tenant model. The authentication and authorization services which provide this model are implemented in the Identity service, Keystone. Every virtual or physical object governed by the OpenStack system exists within a private space referred to as a tenant or project. The latest version of the Keystone API has differentiated itself further to include a higher level construct called a domain. Regardless of the terminology, the innate ability to securely segregate compute, network, and storage resources is the most fundamental capability of the platform. This is what differentiates it from traditional data center virtualization and makes it a private cloud platform.

OpenStack – think again

Today, cloud computing is about **Software as a Service** (**SaaS**), **Platform as a Service**

(**PaaS**), and **Infrastructure as a Service** (**IaaS**). The challenge that has been set by the public cloud is about agility, speed, and service efficiency.

Most companies have expensive IT systems they have developed and deployed over the years, but they are siloed. In many cases, the IT systems are struggling to respond to the agility and speed of the public cloud services that are offered within their own private silos in their own private data center. The traditional data center model and siloed infrastructure might lead to unsustainability.

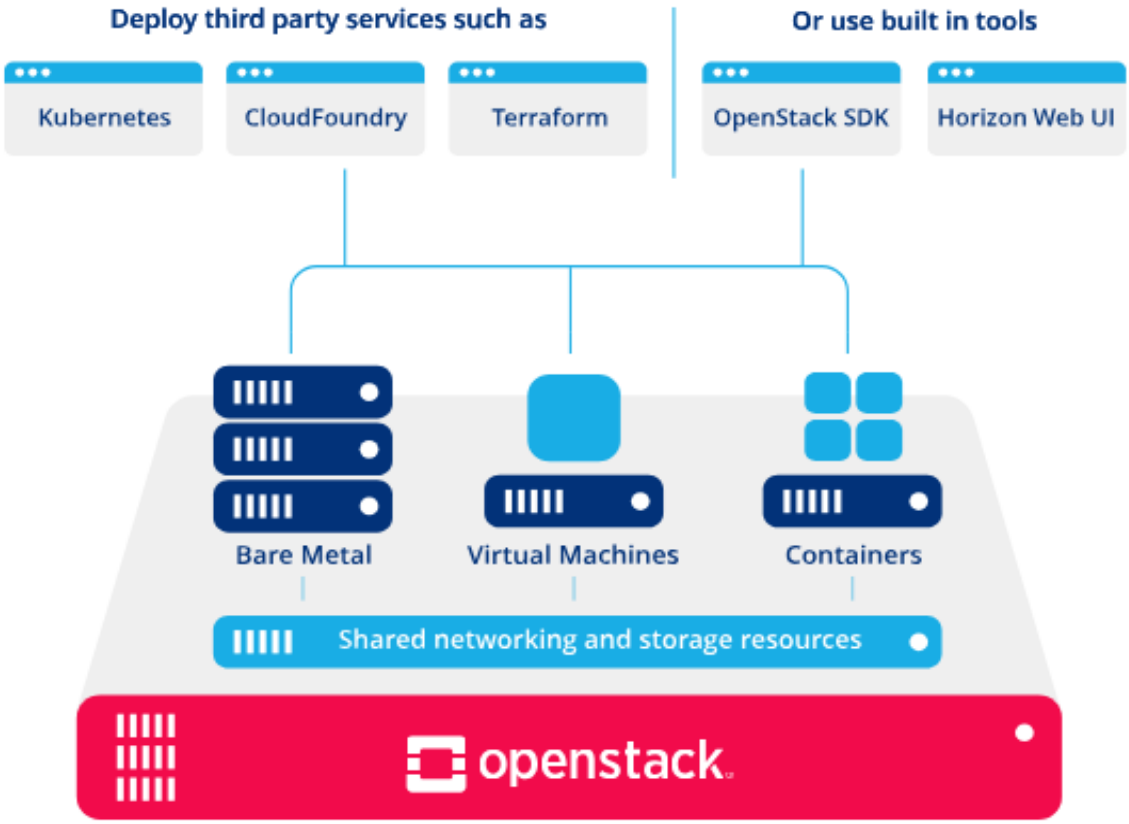
In fact, today's enterprise data center focuses on what it takes to become a next-generation data center. The shift to the new data center generation has evolved the adoption of a model for the management and provision of software. This has been accompanied by a shift from workload isolation in the traditional model to a mixed model. With an increasing number of users utilizing cloud services, the next-generation data centers are able to handle multitenancy.

The traditional one was limited to a single tenancy. Moreover, enterprises today look for scaling down next to scaling up. It is a huge step in the data center technology to shift the way of handling an entire infrastructure.

The big move to a software infrastructure has allowed administrators and operators to deliver a fully automated infrastructure within a minute. The next-generation data center reduces the infrastructure to a single, big, agile, scalable, and automated unit. The end result is that the administrators will have to program the infrastructure.

This is where OpenStack comes into the picture—the next-generation data center operating system. The ubiquitous influence of OpenStack was felt by many big global cloud enterprises such as VMware, Cisco, Juniper, IBM, Red Hat, Rackspace, PayPal, and EBay, to name but a few. Today, many of them are running a very large scalable

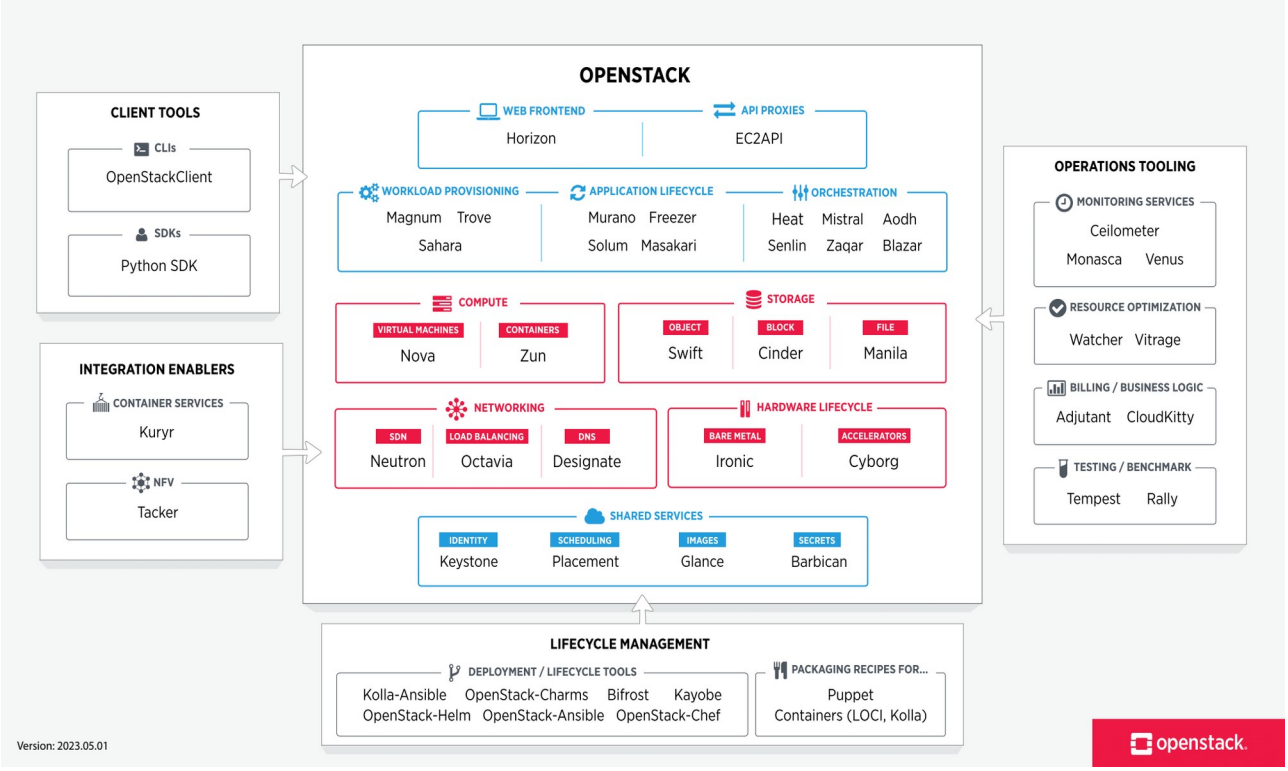
private cloud based on OpenStack in their production environment.



Introducing the OpenStack logical architecture

Before delving into the architecture of OpenStack, we need to refresh or fill gaps, if they do exist, to learn more about the basic concepts and usage of each core component.

In order to get a better understanding on *how it works*, it will be beneficial to first briefly parse *the things that make it work*. Assuming that you have already installed OpenStack or even deployed it in a small or medium-sized environment, let's put the essential core components under the microscope and go a bit further by taking the use cases and asking the question: What is the purpose of such a component?



Keystone

From an architectural perspective, Keystone presents the simplest gservice in the OpenStack composition.

It is the core component that provides identity service and it integrates functions for authentication, catalog services, and policies to register and manage different tenants and users in the OpenStack projects.

The API requests between OpenStack services are being processed by Keystone to ensure that the right user or service is able to utilize the requested OpenStack service.

Keystone performs numerous authentication mechanisms such as username/password as well as a token-authentication-based system.

Additionally, it is possible to integrate it with an existing backend directory such as

Lightweight Directory Access Protocol (**LDAP**) and the **Pluggable Authentication Module** (**PAM**).

A similar real-life example is a city game. You can purchase a gaming day card and profit by playing a certain number of games during a certain period of time. Before you start gaming, you have to ask for the card to get an access to the city at the main entrance of the city game. Every time you would like to try a new game, you must check in at the game stage machine. This will generate a request, which is mapped to a central authentication system to check the validity of the card and its warranty, to profit the requested game. By analogy, the token in Keystone can be compared to the gaming day card except that it does not diminish anything from your request. The identity service is being considered as a central and common authentication system that provides access to the users.

Swift

Although it was briefly claimed that Swift would be made available to the users along with the OpenStack components, it is interesting to see how Swift has empowered what is referred to as **cloud storage**.

Most of the enterprises in the last decade did not hide their fears about a critical part of the IT infrastructure—the storage where the data is held. Thus, the purchasing of expensive hardware to be in the safe zone had become a habit.

There are always certain challenges that are faced by storage engineers and no doubt, one of these challenges includes the task of minimizing downtime while increasing the data availability.

Despite the rise of many smart solutions for storage systems during the last few years,

we still need to make changes to the traditional way.

Make it cloudy! Swift was introduced to fulfill this mission.

We will leave the details pertaining to the Swift architecture for later, but you should keep in mind that Swift is an object storage software, which has a number of benefits:

- No central brain indicates no **Single Point Of Failure** (**SPOF**)
- Curative indicates auto recovery in case of failure
- Highly scalable for large petabytes store access by scaling horizontally
- Better performance, which is achieved by spreading the load over the storage nodes
- Inexpensive hardware can be used for redundant storage clusters

Glance

Swift and Glance are storage systems.

However, the difference between the two is in what they store.

Swift is designed to be an object storage where you can keep data such as virtual disks, images, backup archiving, and so forth, while Glance stores metadata of images.

Metadata can be information such as kernel, disk images, disk format, and so forth.

Do not be surprised that Glance was originally designed to store images. Since the first release of OpenStack included only Nova and Swift (Austin code name October 21, 2010), Glance was integrated with the second release (Bexar code name February 23, 2011).

The mission of Glance is to be an image registry. From this point, we can conclude how OpenStack has paved the way to being more modular and loosely coupled core component model. Using Glance to store virtual disk images is a possibility.

From an architectural level, including more advanced ways to query image information via the Image Service API provided by Glance through an independent image storage backend such as Swift brings more valuable performance and well-organized system core services. In this way, a client (can be a user or an external service) will be able to register a new virtual disk image, for example, to stream it from a highly scalable and redundant store.

Cinder

You may wonder whether there is another way to have storage in OpenStack. Indeed, the management of the persistent block storage is being integrated into OpenStack by using Cinder. Its main capability to present block-level storage provides raw volumes that can be built into logical volumes in the file system and mounted in the virtual machine.

Some of the features that Cinder offers are as follows:

- **Volume management**: This allows the creation or deletion of a volume
- **Snapshot management**: This allows the creation or deletion of a snapshot of volumes
- You can attach or detach volumes from instances
- You can clone volumes
- Volume creation from snapshots is possible via Cinder
- You can copy images to volumes and vice versa

Several storage options have been proposed in the OpenStack core.

Without a doubt, you may be asked this question: What kind of storage will be the best for you? With a decision-making process, a list of pros and cons should be made.

The following is a very simplistic table that describes the difference between the storage types in OpenStack to avoid any confusion when choosing the storage management option for your future architecture design:

Specification	**Storage Type**	
	Object storage	**Block storage**
Performance	-	OK
Database storage	-	OK
Restoring backup data	OK	OK
Setup for volume providers	-	OK
Persistence	OK	OK
Access	Anywhere	Within VM
Image storage	OK	-

It is very important to keep in mind that unlike Glance and Keystone services, Cinder features are delivered by orchestrating volume providers through the configurable setup driver's architectures such as IBM, NetApp, Nexenta, and VMware.

Whatever choice you have made, it is always considered good advice since *nothing is perfect*. If Cinder is proven as an ideal solution or a replacement of the old **nova- volume** service that existed before the **Folsom** release on an architectural level, it is important to know that Cinder has organized and created a catalog of block- based storage devices with several differing characteristics. However, it is obvious if we consider the limitation of commodity storage redundancy and autoscaling.

Eventually, the block storage service as the main core of OpenStack can be improved if a few gaps are filled, such as the addition of values:

- Quality of service
- Replication
- Tiering

The aforementioned Cinder specification reveals its *Non-vendor-lock-in* characteristic, where it is possible to change the backend easily or perform data migration between two different storage backends. Therefore, a better storage design architecture in a Cinder use case will bring a third party into the scalability game. For instance, you can keep in mind that Cinder is essential for our private cloud design, but it misses some capacity scaling features.

Nova

Nova is the most original core component of OpenStack. From an architectural level, it is considered one of the most complicated components of OpenStack.

In a nutshell, Nova runs a large number of requests, which are collaborated to respond to a user request into running VM.

Let's break down the blob image of nova by assuming that its architecture as a distributed application needs orchestration to carry out tasks between different components.

nova-api

The nova-api component accepts and responds to the end user and computes the API calls. The end users or other components communicate with the OpenStack Nova API interface to create instances via OpenStack API or EC2 API.

nova-compute

The nova-compute component is primarily a worker daemon that creates and terminates VM instances via the hypervisor's APIs (XenAPI for XenServer, Libvirt KVM, and the VMware API for VMware).

It is important to depict how such a process works. The following steps delineate this process:

1. Accept actions from the queue and perform system commands such as the launching of the KVM instances to take them out when updating the state in the database.
2. Working closely with nova-volume to override and provide iSCSI or Rados block devices in Ceph.

nova-volume

The nova-volume component manages the creation, attaching, and detaching of *N* volumes to compute instances (similar to Amazon's EBS).

nova-network

The nova-network component accepts networking tasks from the queue and then performs these tasks to manipulate the network (such as setting up bridging interfaces or changing the IP table rules).

nova-scheduler

The nova-scheduler component takes a VM instance's request from the queue
and determines where it should run (specifically which compute server host it should run on).

At an application architecture level, the term *scheduling* or *scheduler* invokes a systematic search for the best outfit for a given infrastructure to improve its performance.

Nova also provides console services that allow end users to access the console of the

virtual instance through a proxy such as nova-console, nova-novncproxy, and nova-consoleauth.

By zooming out the general components of OpenStack, we find that Nova interacts with several services such as Keystone for authentication, Glance for images, and Horizon for the web interface.

For example, the Glance interaction is central; the API process can upload any query to Glance, while nova-compute will download images to launch instances.

Queue

Queue provides a central hub to pass messages between daemons. This is where information is shared between different Nova daemons by facilitating the communication between discrete processes in an asynchronous way.

Any service can easily communicate with any other service via the APIs and queue a service. One major advantage of the queuing system is that it can buffer a large buffer workload. Rather than using an RPC service, a queue system can queue a large workload and give an eventual consistency.

Database

A database stores most of the build-time and runtime state for the cloud infrastructure, including instance types that are available for use, instances in use, available networks, and projects. It is the second essential piece of sharing information in all OpenStack components.

Neutron

Neutron provides a real **Network as a Service** (**NaaS**) between interface devices that are managed by OpenStack services such as Nova. There are various characteristics that should be considered for Neutron:

- It allows users to create their own networks and then attach server interfaces to

them

- Its pluggable backend architecture lets users take advantage of the commodity gear or vendor-supported equipment
- Extensions allow additional network services, software, or hardware to be integrated

Neutron has many core network features that are constantly growing and maturing. Some of these features are useful for routers, virtual switches, and the SDN networking controllers.

Neutron introduces new concepts, which includes the following:

- **Port**: Ports in Neutron refer to the virtual switch connections. These connections are, where instances and network services attached to networks. When attached to the subnets, the defined MAC and IP addresses of the interfaces are plugged into them.
- **Networks**: Neutron defines networks as isolated Layer 2 network segments. Operators will see networks as logical switches that are implemented by the Linux bridging tools, Open vSwitch, or some other software. Unlike physical networks, this can be defined by either the operators or users in OpenStack.
- **Routers**: Routers provide gateways between various networks.
- **Private and floating IPs**: Private and floating IP addresses refer to the IP addresses that are assigned to instances. Private IP addresses are visible within the instance and are usually a part of a private network dedicated to a tenant. This network allows the tenant's instances to communicate when isolated from the other tenants.

    - Private IP addresses are not visible to the Internet.

    - Floating IPs are virtual IPs that Neutron maps instance to private IPs via

Network Access Translation (**NAT**). Floating IP addresses are assigned to an instance so that they can connect to external networks and access the Internet. They are exposed as public IPs, but the guest's operating system has completely no idea that it was assigned an IP address.

In Neutron's low-level orchestration of Layer 1 through Layer 3, components such as IP addressing, subnetting, and routing can also manage high-level services. For example, Neutron provides **Load Balancing as a Service** (**LBaaS**) utilizing HAProxy to distribute the traffic among multiple compute node instances.

The Neutron architecture

There are three main components of Neutron architecture that you ought to know in order to validate your decision later with regard to the use case for a component within the new releases of OpenStack:

- **Neutron-server**: It accepts the API requests and routes them to the appropriate neutron-plugin for its action
- **Neutron plugins and agents**: They perform the actual work such as the plugging in or unplugging of ports, creating networks and subnets, or IP addressing.
- **Queue**: This routes messages between the neutron-server and various agents as well as the database to store the plugin state for a particular queue

Neutron is a system that manages networks and IP addresses. OpenStack networking ensures that the network will not be turned into a bottleneck or limiting factor in a cloud deployment and gives users real self-service, even over their network configurations.

Another advantage of Neutron is its capability to provide a way for organizations to relieve stress within the network of cloud environments and to make it easier to deliver NaaS in the cloud. It is designed to provide a plugin mechanism that will provide an

option for the network operators to enable different technologies via the Neutron API.

It also lets its tenants create multiple private networks and control the IP addressing on them.

As a result of the API extensions, organizations have additional control over security and compliance policies, quality of service, monitoring, and troubleshooting, in addition to paving the way to deploying advanced network services such as firewalls, intrusion detection systems, or VPNs.

Horizon

Horizon is the web dashboard that pools all the different pieces together from your OpenStack ecosystem.

Horizon provides a web frontend for OpenStack services. Currently, it includes all the OpenStack services as well as some incubated projects. It was designed as a stateless and data-less web application—it does nothing more than initiating actions in the OpenStack services via API calls and displaying information that OpenStack returns to the Horizon. It does not keep any data except the session information in its own data store.

It is designed to be a reference implementation that can be customized and extended by operators for a particular cloud. It forms the basis for several public clouds—most notably the HP Public Cloud and at its heart, is its extensible modular approach to construction.

Horizon is based on a series of modules called **panels** that define the interaction of each service. Its modules can be enabled or disabled, depending on the service availability of the particular cloud. In addition to this functional flexibility, Horizon is easy to style with **Cascading Style Sheets** (**CSS**).

Most cloud provider distributions provide a company's specific theme for their dashboard implementation.